

Sprawozdanie z laboratorium:
Sztuczna inteligencja w grach

Część 1. Zachowanie botów: optymalizacja vs. uczenie vs. heurystyka

10 kwietnia 2026

Prowadzący: dr hab. inż. Maciej Komosiński, prof. PP

Autorzy: **Damian Krzyżelewski** 151796 damian.krzyzelewski@student.put.poznan.pl

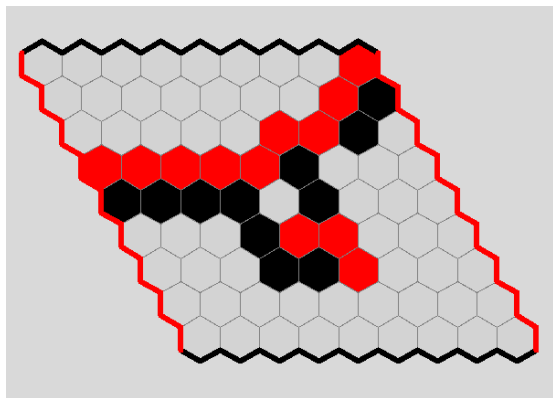
Zajęcia: co drugi poniedziałek (parzyste), 15:10 - 16:40.

Oświadczam/y, że niniejsze sprawozdanie i towarzyszący mu kod źródłowy zostały przygotowane wyłącznie przez powyższych autora/ów, a wszystkie elementy pochodzące z innych źródeł zostały odpowiednio zaznaczone i są cytowane w bibliografii.

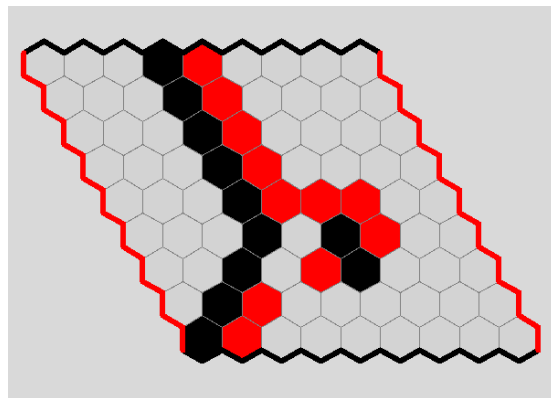
1 Wstęp, zasady i podstawowe strategie wybranej gry

Wybrano grę **Hex**. Rozgrywka toczy się na planszy o rozmiarze 9×9 , składającej się z sześciokątnych pól. Hex jest klasyczną strategiczną grą planszową o sumie zerowej, z pełną informacją i bez elementów losowych. Ze względów topologicznych w grze nie występują remisy - każda partia kończy się zwycięstwem jednej ze stron [2].

W grze bierze udział dwóch graczy, z których każdy ma przypisane dwa przeciwległe brzegi planszy (oznaczane różnymi kolorami). Celem gracza jest utworzenie nieprzerwanego łańcucha swoich kamieni, łączącego jego krawędzie poprzez sąsiadujące pola. Gracze wykonują ruchy naprzemiennie (zaczynając od gracza czarnego), zajmując po jednym wolnym polu na planszy. Realizacja własnego celu jednocześnie prowadzi do blokowania ścieżek przeciwnika. Przykładowa plansza po zakończeniu rozgrywki i zwycięstwie gracza czerwonego i czarnego została przedstawiona na Rys.1.



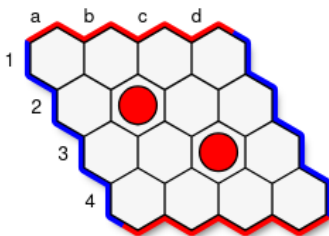
(a) Zwycięstwo gracza czerwonego: łańcuch łączy krawędzie lewą i prawą.



(b) Zwycięstwo gracza czarnego: łańcuch łączy krawędzie górną i dolną.

Rysunek 1: Przykładowe plansze po zakończeniu rozgrywki na planszy 9×9 .

Ważnym elementem mechaniki gry, mającym na celu zbalansowanie rozgrywki, jest tzw. reguła zamiany (ang. *swap rule* lub *pie rule*). W klasycznym wariancie gry pierwszy gracz posiada znaczącą przewagę. Reguła zamiany niweluje ten problem, dając drugiemu graczowi wybór po pierwszym ruchu przeciwnika: może on przejąć wykonany ruch jako własny (dokonać zamiany) lub zrezygnować z tej opcji i wykonać standardowy ruch [4]. Mechanizm ten wymusza na gracz rozpoczynającym wykonanie możliwie najbardziej neutralnego i zbalansowanego ruchu otwierającego. Zbyt silne otwarcie może skłonić przeciwnika do zamiany, natomiast zbyt słabe pozwala mu uzyskać przewagę bez konieczności jej zastosowania.



Rysunek 2: Przykład wzorca wewnętrznego typu *most* (ang. *bridge*).

Ważnym aspektem strategicznym w grze są tzw. wzorce wewnętrzne [3] (ang. *interior templates*). Są to zdefiniowane konfiguracje własnych kamieni oraz pustych pól, które matematycznie gwarantują połączenie, niezależnie od tego, jak odpowie przeciwnik. Najprostszy i najczęściej stosowany wzorek tego typu jest *most*. Tworzą go dwa kamienie tego samego koloru, pomiędzy którymi znajdują się dokładnie dwa puste pola posiadające wspólną krawędź, jak widać na Rys. 2. Jeżeli przeciwnik zajmie jedno z tych pustych pól, gracz zawsze może zająć drugie, domykając połączenie pomimo próby bloku.

Zrozumienie topologii planszy i wzorców jest również kluczowe w defensywie. Podstawowym błędem, często popełnianym przez początkujących graczy, jest blokowanie przeciwnika poprzez dostawianie kamieni bezpośrednio przy czole jego łańcucha. Znacznie skuteczniejszą strategią jest blokowanie na dystans. Umieszczenie kamienia blokującego w pewnym oddaleniu od rozwijającej się ścieżki oponenta zmusza go do zmiany planów i daje broniącemu się graczowi dodatkowy czas na zbudowanie solidnego "muru", zanim łańcuch przeciwnika faktycznie do niego dotrze [1].

2 Podejście 1: Heurystyka

2.1 Faza otwarcia i reguła zamiany

Niezależnie od wersji heurystyki, zaimplementowano wspólną, dosyć prostą wiedzę dotyczącą pierwszych ruchów w grze:

- **Otwarcie:** W 80% przypadków bot wybiera bezpieczne, zbalansowane pozycje startowe (aby zniechęcić przeciwnika do użycia *swap rule*). W 20% przypadków stosuje strategię agresywną, zajmując losowe pola w ścisłym centrum planszy, które w przypadku braku zamiany dają mu przewagę.
- **Swap rule:** Jeśli bot wykonuje drugi ruch w grze, analizuje pozycję kamienia przeciwnika. Jeśli znajduje się on w promieniu dwóch pól od idealnego środka planszy, bot automatycznie aktywuje regułę zamiany i przejmuje ten kamień.

2.2 Agent Bazowy (Blind Heuristic Agent)

Pierwsza wersja bota (później oznaczana jako **Blind Heuristic Agent**) opierała się na najprostszym i najbardziej oczywistym założeniu: zajmować pola, które tworzą najkrótszą ścieżkę między własnymi krawędziami. W tym celu wykorzystano przeszukiwanie wszerz z wagami 0-1 (0-1 BFS). Koszt przejścia przez własny kamień wynosił 0, przez pole puste 1, a pola zajęte przez przeciwnika traktowano jako niedostępne (koszt nieskończony). Bot sumował dystans z danego pustego pola do obu swoich krawędzi, a w przypadku remisów preferował pola znajdujące się bliżej centrum. Bot ten poprawnie dążył do celu, jednak jego główną wadą był całkowity brak gry defensywnej - zupełnie ignorował ścieżki budowane przez przeciwnika. Co więcej, jego strategia ofensywna była bardzo naiwna: agent dążył do bezpośredniego łączenia kamieni bok w bok, nie potrafiąc wykorzystywać bardziej zaawansowanych topologicznie struktur, takich jak mosty, które są znacznie trudniejsze do zablokowania.

2.3 Agent Ulepszony (Heuristic Agent)

Aby wyeliminować słabości wersji bazowej, została stworzona kolejna wersja bota (później oznaczana jako **Heuristic Agent**). Agent ten oblicza najkrótsze ścieżki zarówno dla siebie, jak i dla oponenta. Dla każdego potencjalnego ruchu bot generuje ocenę w postaci krotki wartości, na podstawie której podejmuje ostateczną decyzję. Kluczowym elementem oceny jest *critical_score* - wartość zdefiniowana jako minimum z długości własnej ścieżki oraz ścieżki przeciwnika (powiększonej o niewielką stałą, aby delikatnie preferować własny atak nad obronę). Dzięki temu zabiegowi, bot potrafi płynnie przełączać się między realizacją własnego celu, a bezwzględnym blokowaniem przeciwnika, gdy ten jest bliski zwycięstwa. Należy jednak zaznaczyć, że mimo wprowadzonych usprawnień, podejście to nadal obciążone jest ograniczeniami wynikającymi z zastosowania klasycznego algorytmu najkrótszej ścieżki. Defensywa bota opiera się na bezpośrednim odcinaniu wyliczonej trasy przeciwnika, przez co agent nie potrafi stosować efektywniejszego strategicznie blokowania na dystans. Podobnie w fazie ataku, konstrukcja własnego łańcucha realizowana jest w sposób naiwny tak jak wykonywał to **Blind Heuristic Agent**.

2.4 Próby wykorzystania wiedzy dziedzinowej

W ramach prac podjęto również próby zaimplementowania bardziej złożonych heurystyk, będących zbiorem ręcznie projektowanych reguł, takich jak priorytetyzowanie tworzenia mostów, czy blokowanie wyprzedzające na dystans. Podejście to ostatecznie zostało odrzucone, ponieważ problemem okazało się właściwe ręczne dobieranie wag poszczególnych składowych oceny danego ruchu dla wielu reguł. Próby ulepszenia jednego aspektu gry zazwyczaj psuły zachowanie bota w innych sytuacjach na planszy, przez

co przykładowo bot rozsiewał po planszy niepołączone mosty, zapominając o konieczności ich ostatecznego domknięcia i uformowania spójnej ścieżki. Ostatecznie okazało się, że znacznie prostsze podejście zastosowane w *Heuristic Agent* z łatwością pokonywało ręcznie strojone wersje bota.

3 Podejście 2: Optymalizacja

Drugie podejście jest rozwinięciem idei wykorzystania wiedzy dziedzinowej z punktu 2.4. Zdefiniowano zestaw cech planszy, a znalezienie optymalnych wag dla tych cech powierzono algorytmowi **CMA-ES** (ang. *Covariance Matrix Adaptation Evolution Strategy*).

3.1 Reprezentacja osobnika i ocena ruchu

Osobnik w populacji algorytmu CMA-ES był reprezentowany jako wektor 12 liczb rzeczywistych. Wartości te stanowiły wagi funkcji oceny, która w każdej turze przypisywała wartość wszystkim legalnym ruchom. Ocena ruchu była obliczana jako kombinacja liniowa wektora wag (pochodzących od osobnika) oraz wektora cech wyekstrahowanych z aktualnego stanu planszy. Bot wybierał ruch o najwyższej uzyskanej wartości funkcji oceny.

Do ewaluacji pojedynczego ruchu wyznaczano 12 znormalizowanych cech środowiska. Szczegółowe zestawienie oraz interpretację wykorzystanych cech przedstawiono w Tabeli 1.

Nazwa cechy (w kodzie)	Opis
<code>my_path_score</code>	Znormalizowana długość najkrótszej ścieżki naszego gracza do celu po wykonaniu ruchu. Krótsza ścieżka oznacza lepszą pozycję.
<code>opp_path_score</code>	Znormalizowana długość najkrótszej ścieżki przeciwnika do celu.
<code>path_adv</code>	Różnica między długością naszej ścieżki a ścieżką przeciwnika - pokazuje kto ma przewagę.
<code>center_score</code>	Określa jak blisko środka planszy znajduje się pole. Pola bliżej środka mają wyższą wartość.
<code>friend_neighbors</code>	Liczba sąsiednich pól zajętych przez nasze kamienie
<code>enemy_neighbors</code>	Liczba sąsiednich pól zajętych przez kamienie przeciwnika.
<code>touched_components</code>	Liczba naszych rozłącznych grup kamieni, które zostaną połączone przez ten ruch.
<code>component_size_after</code>	Wielkość powstałej grupy kamieni po ruchu w stosunku do wielkości planszy.
<code>bridge_support</code>	Liczba nowych mostów tworzonych przez ruch.
<code>bridge_block</code>	Liczba mostów przeciwnika zablokowanych przez ten ruch.
<code>liberties</code>	Liczba pustych pól wokół zagrane ruchu.
<code>swap_bias</code>	Cecha używana tylko przy pierwszym ruchu drugiego gracza do decyzji o użyciu reguły swap.

Tabela 1: Cechy używane do oceny ruchów.

3.2 Funkcja przystosowania i Curriculum Learning

Funkcją przystosowania był odsetek wygranych partii. Ze względu na charakter gry, ocenianie osobników wyłącznie przeciwko jednemu, stałemu przeciwnikowi prowadzi do przetrenowania i wyuczenia się bardzo wąskiej strategii, a zastosowanie od razu bardzo dobrych przeciwników doprowadziłoby do sytuacji, że większość osobników przegrywa praktycznie wszystko (problemem rzadkiej nagrody). Aby temu zapobiec, zastosowano technikę stopniowego zwiększania trudności przeciwników (ang. *Curriculum Learning*). Istotnym elementem procesu był mechanizm *Hall of Fame*. Przechowywał on najlepsze osobniki z poprzednich

generacji, które były wykorzystywane jako przeciwnicy w trakcie ewaluacji. Proces optymalizacji podzielono na trzy fazy o rosnącym poziomie trudności:

- **Faza początkowa (do 30% generacji):** szybkie odrzucenie najsłabszych osobników. Przeciwnikiem był w 60% agent losowy oraz w 40% *Heuristic Agent*.
- **Faza środkowa (30%-70% generacji):** udział agenta losowego zmniejszono do 20%. Wprowadzono mechanizm *Hall of Fame* (HoF), z którymi rozgrywano 30% gier. Pozostałe 50% gier rozgrywano przeciwko *Heuristic Agent*.
- **Faza końcowa (od 70% generacji):** całkowicie usunięto agenta losowego. Osobniki były oceniane przeciwko trudniejszym przeciwnikom: *Heuristic Agent* (50% gier) oraz agentom z HoF (50%).

Ostatecznym wynikiem procesu był osobnik osiągający najwyższy wskaźnik *win-rate* w trzeciej, najtrudniejszej fazie optymalizacji.

4 Podejście 3: Uczenie ze wzmocnieniem

Najbardziej zaawansowanym podejściem było zastosowanie metod głębokiego uczenia ze wzmocnieniem (ang. *Deep Reinforcement Learning*). Ponieważ w grze Hex liczba dostępnych pól stale maleje, zastosowano algorytm **PPO** z mechanizmem maskowania akcji (ang. *Maskable Proximal Policy Optimization*) z biblioteki *Stable Baselines3* (SB3). Dzięki temu agent z góry ignoruje niedozwolone ruchy (np. próby postawienia kamienia na zajęтым polu), co pozwala zaoszczędzić czas i znacząco przyspiesza proces uczenia.

Dodatkowo, aby ułatwić naukę, zastosowano ujednolicenie perspektywy, tzn. agent zawsze dąży do połączenia góry z dołem, co pozwala sieci skupić się na jednej strategii zamiast uczyć się dwóch kierunków naraz. W praktyce realizowane jest to poprzez zamianę współrzędnych rzędów i kolumn (transpozycję), co pozwala sieci zawsze analizować sytuację w pionie. Po podjęciu decyzji przez model, współrzędne są przywracane do oryginalnego układu, aby ruch trafił na właściwe miejsce na rzeczywistej planszy.

Kolejnym usprawnieniem jest wykorzystanie symetrii obrotowej planszy o 180° jako formy augmentacji danych. Dla każdego stanu planszy sieć otrzymuje jego "bliźniaczy" odpowiednik. Pozwala to na dwukrotne zwiększenie liczby przykładów uczących bez konieczności rozgrywania nowych partii.

4.1 Reprezentacja stanu i architektura sieci

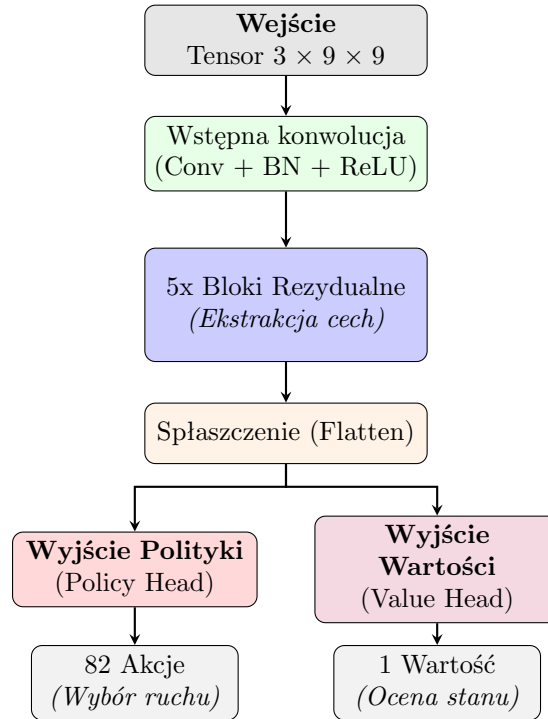
Aby sieć neuronowa mogła skutecznie analizować sytuację na planszy, stan środowiska był przekazywany w postaci tensora o wymiarach $3 \times 9 \times 9$. Trzy kanały wejściowe kodowały odpowiednio:

- pozycje własnych kamieni gracza,
- pozycje kamieni przeciwnika,
- puste pola na planszy.

Do ekstrakcji cech wykorzystano głęboką konwolucyjną sieć neuronową opartą na architekturze **ResNet**. Składała się ona z początkowej warstwy konwolucyjnej oraz pięciu bloków rezydualnych (ang. *residual blocks*) po 128 filtrów każdy. Po przetworzeniu danych przez bloki rezydualne i warstwę flatten, model przechodzi do fazy decyzyjnej. Wyjście sieci stanowi warstwa liniowa, która generuje wartości dla 82 możliwych akcji (81 pól planszy oraz opcja *swap rule*). Szczegółowy schemat przedstawia Rys. 3.

4.2 Faza 0: Imitation Learning (uczenie nadzorowane)

W tej fazie model uczył się naśladować ruchy gotowych algorytmów heurystycznych (*Behavioral Cloning*) na zbiorze 8000 rozegranych partii. Jako funkcję straty wykorzystano entropię krzyżową. Dzięki tak przygotowanym wagom początkowym, właściwy proces uczenia ze wzmocnieniem mógł od razu skupić się na optymalizacji strategii i poszukiwaniu ruchów lepszych niż te wynikające z prostych heurystyk, co znacząco przyspieszyło proces uczenia modelu.



Rysunek 3: Architektura sieci PPO

4.3 System ligi i zapobieganie zapomniani

Kluczowym elementem implementacji środowiska treningowego był stworzony moduł ligi. Uczenie agenta wyłącznie poprzez rozgrywki z samym sobą (ang. *Self-Play*) niesie ryzyko wystąpienia zjawiska katastrofalego zapomnienia (ang. *catastrophic forgetting*) - agent może wyuczyć się wąskiej strategii pokonującej jego aktualną iterację, zapominając przy tym podstawowych zasad obrony przed najprostszymi atakami. Aby temu zaradzić, moduł ligi dynamicznie losował przeciwników dla każdej partii według następującego rozkładu:

- **50% - Agenci heurystyczni:** Zapewniali oni stabilny punkt odniesienia, wymuszając na modelu zachowanie fundamentalnych zdolności defensywnych i ofensywnych przeciwko prostym graczom.
- **25% - Historyczne modele:** Agent rywalizował ze swoimi starszymi, zapisanymi wcześniej wersjami. Mechanizm ten gwarantował ciągły progres i uniemożliwiał uwstecznianie się strategii.
- **25% - Aktualny model (Self-Play):** Rywalizacja z samym sobą.

4.4 Proces treningu (Curriculum Learning)

Właściwy trening z wykorzystaniem algorytmu Maskable PPO podzielono na dwie fazy o rosnącym poziomie trudności, kontynuując ideę *Curriculum Learning*:

- **Rozgrzewka krytyka (Critic Warmup):** Przez pierwsze 200 tysięcy kroków wagi wstępnie wytrenowanego ekstraktora cech oraz głowy decyzyjnej aktora były zamrożone. Sieć uczyła się wyłącznie oceniać wartość danego stanu. Przeciwnikiem na tym etapie był tylko agent heurystyczny. Celem tego zabiegu było nauczenie krytyka poprawnej oceny stanów planszy przed odmrożeniem aktora, aby jego początkowo losowe błędy nie zniszczyły wag uzyskanych w procesie BC.
- **Trening docelowy:** Po rozgrzewce odmrożono wszystkie wagi modelu i uruchomiono pełny system ligi. Faza ta trwała 3.5 milionów kroków środowiska (czyli około 170 000 rozegranych partii zakładając średnio 20 ruchów agenta na partię). Zastosowano harmonogram liniowego spadku współczynnika prędkości uczenia (ang. *learning rate schedule*), co pozwoliło na precyzyjne dostrojenie parametrów modelu w końcowych fazach treningu. Proces ten mógł być dłuższy, ale na tym etapie model wypracował strategię na tyle zaawansowaną, że był w stanie powtarzalnie pokonywać ludzkiego gracza

(autora projektu), wykazując się przy tym skutecznym blokowaniem i tworzeniem nieoczywistych połączeń. Uznano, że uzyskany poziom inteligencji modelu jest w pełni wystarczający do realizacji celów projektu.

5 Porównanie wyników i analiza

5.1 Skuteczność botów - turniej

Aby obiektywnie ocenić skuteczność zaimplementowanych rozwiązań, przeprowadzono turniej, w którym agenci rozgrywali między sobą (w systemie "każdy z każdym") po 1000 partii - 500 jako czarny i 500 jako czerwony. Zbiorcze statystyki z tych rozgrywek zestawiono na Rys. 4, natomiast szczegółowe rezultaty bezpośrednich starć przedstawia macierz na Rysunku 5.

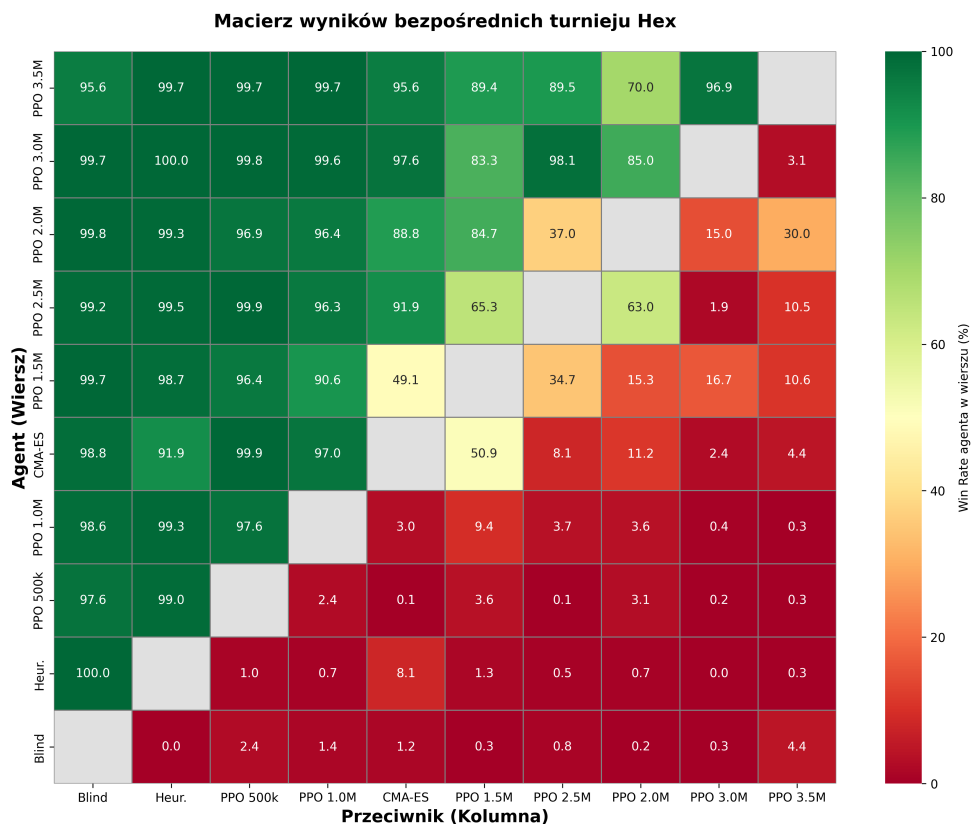
WYNIKI TURNIEJU					
Miejsce / Przeciwnik	Bot	Wygrane	Przegrane	Rozegrane	Win Rate (%)
▷ #1	PPO_3500000_steps	8361	639	9000	92.9%
▷ #2	PPO_3000000_steps	7662	1338	9000	85.13%
▷ #3	PPO_2000000_steps	6479	2521	9000	71.99%
▷ #4	PPO_2500000_steps	6275	2725	9000	69.72%
▷ #5	PPO_1500000_steps	5118	3882	9000	56.87%
▷ #6	CMA-ES	4646	4354	9000	51.62%
▷ #7	PPO_1000000_steps	3159	5841	9000	35.1%
▷ #8	PPO_500000_steps	2064	6936	9000	22.93%
▷ #9	Heuristic Agent	1126	7874	9000	12.51%
▷ #10	Blind Heuristic Agent	110	8890	9000	1.22%

Rysunek 4: Tabela wyników rozgrywek między botami

Model PPO 3.5M (trenowany przez 3.5 mln kroków) okazał się bezapelacyjnym zwycięzcą turnieju osiągając całkowity współczynnik wygranych na poziomie 92.9%. Analizując ogólne zestawienie wyraźnie widać dodatnią korelację między długością procesu uczenia a ostateczną skutecznością modeli opartych na algorytmie PPO. Z reguły każda kolejna wersja agenta radziła sobie coraz lepiej w turnieju, ale w dokładnych statystykach pojedynków "każdy z każdym" można zauważyć pewne odstępstwa. Model PPO 2.0M uplasował się w ogólnym rankingu wyżej (trzecie miejsce z wynikiem 71.99% zwycięstw) niż jego dłużej trenowany odpowiednik PPO 2.5M. Patrząc na bezpośrednie starcia z mistrzowską wersją 3.5M w macierzy można dostrzec, że agent PPO 2.0M przegrywa z nią w 70% partii, podczas gdy nowsza, teoretycznie silniejsza iteracja 3.0M ulega mistrzowi w aż 96.9% przypadków. Takie odstępstwa można wytłumaczyć specyfiką uczenia ze wzmocnieniem i modułu ligi w treningu docelowym. Podczas treningu wersji 3.5M, agent 2.0M mógł być po prostu rzadziej losowany jako przeciwnik z puli modeli historycznych. Ponadto, iteracja 2.0M mogła mieć w tamtym momencie wypracowany bardzo nietypowy styl gry, na który ostateczny model nie zdążył się w pełni uodpornić. Może to być również dobrym wytłumaczeniem, dlaczego agent 2.5M osiągnął gorszy win-rate w ogólnym rankingu. Możliwe, że znajdował się on w "fazie przejściowej", w której odszedł od skutecznej, ale specyficznej strategii wersji 2.0M, i dopiero ewoluował w kierunku nowej, docelowo optymalnej koncepcji, która zaczęła się stabilizować w wersjach 3.0M oraz 3.5M.

Analizując skuteczność pozostałych zaimplementowanych podejść, należy docenić agenta stworzonego z użyciem metody optymalizacji. Był on w stanie konkurować jak równy z równym z modelem uczenia ze wzmocnieniem na niemalże półmetku procesu nauki. CMA-ES wypracował minimalną bezpośrednią przewagę, wygrywając 50.9% rozegranych partii przeciwko agentowi PPO trenowanemu przez 1.5 miliona kroków, jednocześnie plasując się w rankingu ogólnym tuż za nim, na szóstym miejscu z wynikiem 51.62% zwycięstw.

Na przeciwnym biegunie, zgodnie z oczekiwaniami, znalazły się boty heurystyczne oparte na algorytmie wyznaczania najkrótszej ścieżki. Zarówno Heuristic Agent, jak i Blind Heuristic Agent odnotowały niskie współczynniki zwycięstw w całym turnieju - wynoszące odpowiednio 12.51% oraz zaledwie 1.22%. Jednak dogłębna analiza macierzy wyników bezpośrednich pokazuje, że swój całkowity współczynnik zwycięstw na poziomie 12.51% Heuristic Agent zawdzięcza niemal wyłącznie absolutnej dominacji nad wersją bazową - wygrał on równe 100% partii przeciwko Blind Heuristic Agent. Jednak znacznie ciekawszym zjawiskiem, które można zaobserwować jest to, że najsłabszy w zestawieniu Blind Heuristic Agent swój



Rysunek 5: Macierz wyników bezpośrednich w turnieju botów Hex. Wartości w komórkach przedstawiają procent zwycięstw agenta znajdującego się w danym wierszu przeciwko agentowi w danej kolumnie.

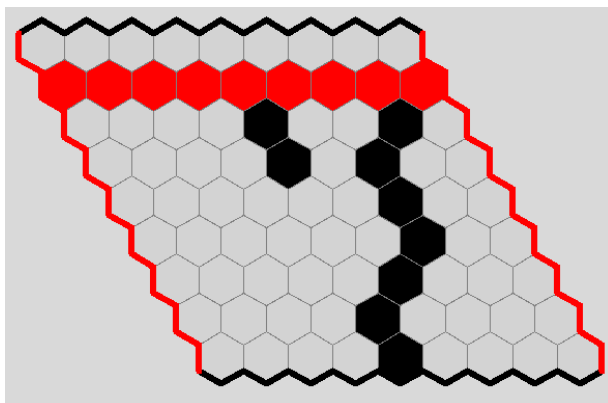
najwyższy procent zwycięstw (4.4%) odnotował przeciwko absolutnemu zwycięzcy turnieju, modelowi PPO 3.5M. Zjawiska tego nie widać w poprzednich fazach uczenia - słabsze modele, takie jak PPO 2.5M czy 3.0M, radziły sobie z tymi naiwnymi atakami niemal bezbłędnie (odnosząc porażki w zaledwie 0.2% - 0.3% partii). Tę z pozoru nielogiczną anomalię można wytłumaczyć zjawiskiem "przeuczenia" na zaawansowanych przeciwnikach. Agent PPO w końcowej fazie treningu był optymalizowany w większości do obrony przed złożonymi i przemyślanymi atakami. Można przypuszczać, że zaczął trochę bagatelizować absurdalnie proste i naiwne ataki najsłabszego bota, przez co niespodziewanie przegrał w aż 44 starciach na 1000.

5.2 Analiza zachowania

5.2.1 Boty heurystyczne

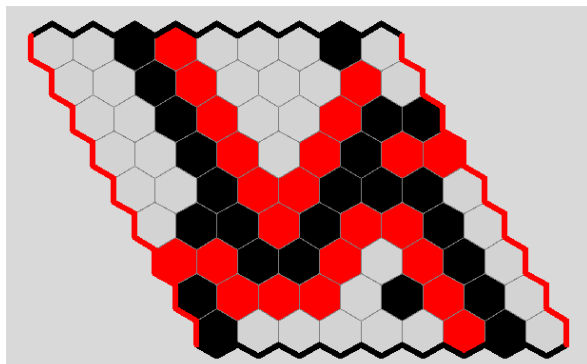
Przykład rozgrywki zaprezentowany na Rysunku 6 pokazuje brak użyteczności agenta bazowego (**Blind Heuristic Agent**) w praktycznej rozgrywce z graczem ludzkim ze względu na jego fundamentalne braki w grze defensywnej i skrajną przewidywalność. W ramach tego projektu pełni on wyłącznie rolę prostego punktu odniesienia (ang. *baseline*).

Z kolei bot heurystyczny (**Heuristic Agent**) stanowi dobre odzwierciedlenie początkującego gracza ludzkiego. Choć w przeciwieństwie do wersji bazowej potrafi on już reagować na ruchy oponenta, robi to w sposób naiwny, opierając swoją defensywę na bezpośrednim odcinaniu ścieżki. Algorytm ten nie wykazuje głębszego zrozumienia topologii planszy. Nie potrafi stosować blokowania na dystans ani celowo budować zaawansowanych, odpornych na przerwanie struktur. Mimo tych ograniczeń, jego zachowanie jest na tyle spójne i ukierunkowane na cel, że doskonale sprawdza się on w roli podstawowego przeciwnika o niskim poziomie trudności, oferując odpowiednie środowisko treningowe dla osób, które dopiero uczą się grać.

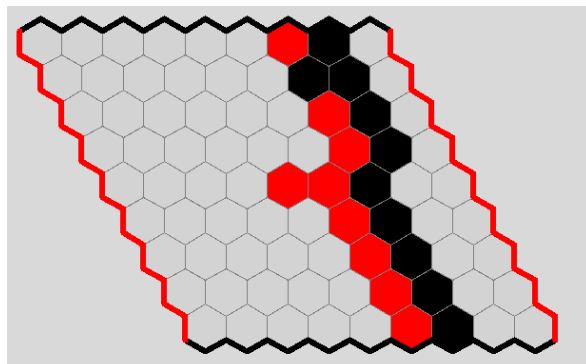


Rysunek 6: Przykład naiwnego zachowania Blind Heuristic Agent (czarne kamienie). Gracz ludzki (czerwone kamienie) bez problemu zablokował ścieżkę bota prostą linią poziomą wygrywając w ten sposób partię.

Typowe zachowanie tego agenta ilustrują przykładowe dwie rozgrywki zaprezentowane na Rysunku 7. Wyraźnie widać na nim, że stosując naiwne podejście oparte na bezpośrednim blokowaniu (lewa strona), gracz napotyka na znaczny opór ze strony algorytmu, a partia staje się zacięta i gęsta od kamieni. Z kolei w momencie, gdy człowiek zaczyna grać w sposób bardziej przemyślany, wykorzystując wiedzę topologiczną i wzorce wewnętrzne takie jak mosty (prawa strona), prosta heurystyka bota okazuje się całkowicie bezradna próbując jakkolwiek zablokować rywala. Na rysunku 8 zaprezentowano zaimplementowane w heurystykach zasady otwarcia i kiedy wykonywać zamianę.



(a) Strategia naiwna: brak mostów i bezpośrednie blokowanie.

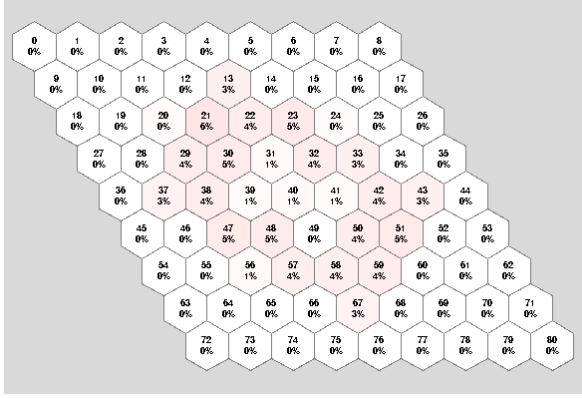


(b) Strategia zaawansowana: łatwa wygrana dzięki mostom i blokowaniu na dystans.

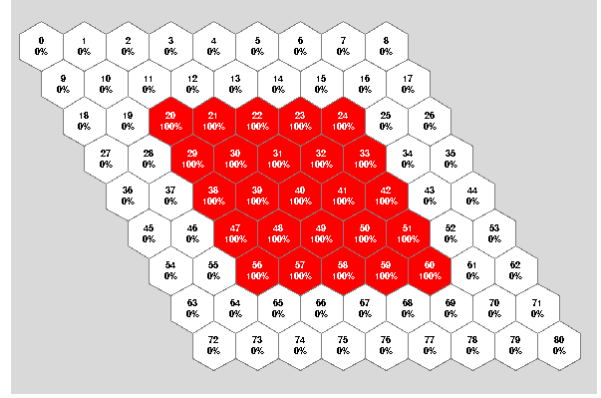
Rysunek 7: Porównanie dwóch zwycięskich partii gracza ludzkiego (kolor czarny) przeciwko **Heuristic Agent** (kolor czerwony). Rysunek po lewej przedstawia partię rozegraną w sposób naiwny bez użycia wzorców, co doprowadziło do zaciętej walki i gęstego wypełnienia planszy. Rysunek po prawej ukazuje w pełni kontrolowane zwycięstwo osiągnięte poprzez świadome stosowanie mostów.

5.2.2 Bot stworzony przy pomocy algorytmu optymalizacji

Analiza wektora wag zoptymalizowanego przez algorytm ewolucyjny CMA-ES (przedstawionego w tabeli 2), ujawnia bardzo specyficzną i niespodziewaną strategię gry, która znacząco odbiega od klasycznego podejścia i potwierdza wnioski z praktycznych rozgrywek z tym agentem. Przede wszystkim, model wykazuje skrajnie defensywne nastawienie. Waga cechy oceniającej ścieżkę przeciwnika ($opp_path_score = 24.64$) jest aż trzykrotnie wyższa niż waga przypisana własnej ścieżce ($my_path_score = 8.06$). Oznacza to, że bot na pierwszym miejscu stawia blokowanie oponenta, a realizację własnego połączenia traktuje jako cel drugorzędny. Największym zaskoczeniem jest jednak silnie ujemna waga przypisana do cechy cen-



(a) Rozkład otwarć agenta

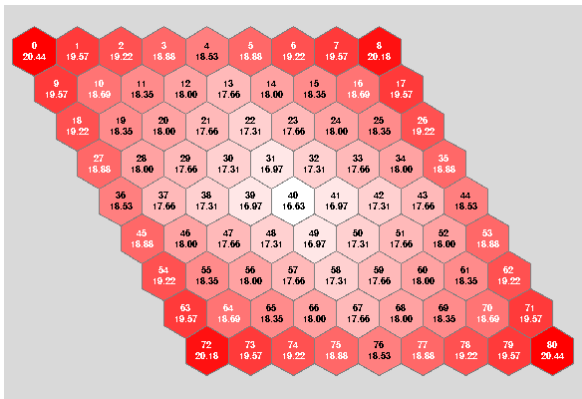


(b) Decyzje reguły zamiany (swap)

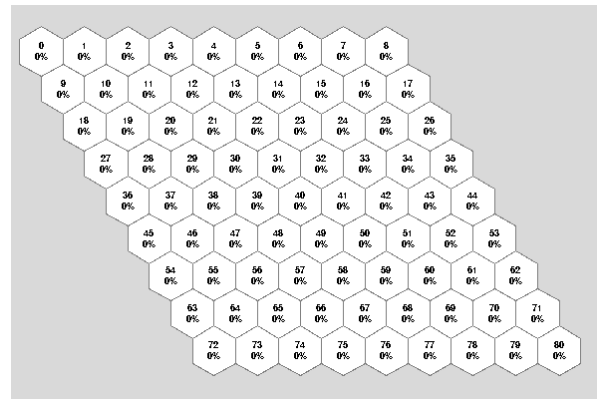
Rysunek 8: Analiza fazy otwarcia i reguły swap dla Heuristic Agent.

Nazwa cechy	Wartość wagi
my_path_score	8.0644
opp_path_score	24.6421
path_adv	-3.6588
center_score	-5.5106
friend_neighbors	-4.4368
enemy_neighbors	1.0500
touched_components	6.7797
component_size_after	8.3478
bridge_support	1.9727
bridge_block	-4.0666
liberties	-1.5874
swap_bias	0.1151

Tabela 2: Wartości wag poszczególnych cech zoptymalizowane przez algorytm CMA-ES.



(a) Oceny pól przy otwarciu



(b) Decyzje reguły zamiany (swap)

Rysunek 9: Analiza fazy otwarcia i reguły swap dla agenta CMA-ES.

ter_score (-5.51). W klasycznej strategii gry w Hex zajmowanie centrum planszy jest uważane za kluczowe dla odniesienia zwycięstwa. Algorytm CMA-ES "zdecydował" odwrotnie - aktywnie unika on środkowych pól, preferując grę blisko krawędzi. Potwierdza to wygenerowana mapa ocen przy otwarciu (Rysunek 9a), na której centralne pola posiadają najniższą wartość, podczas gdy najwyżej oceniane są rogi planszy. Tłumaczy to również zaskakujące zachowanie bota związane z regułą zamiany. Jak widać na Rysunku 9b, agent CMA-ES w ogóle nie korzysta z tej opcji (0% we wszystkich polach).

Wysoka skuteczność w grze z innymi botami potwierdzona w turnieju nie przekłada się jednak na konfrontację z człowiekiem (autorem projektu). Trenując na ograniczonej puli sztucznych przeciwników, bot odnalazł specyficzne lokalne optimum. Skrajnie defensywna gra i budowanie własnej ścieżki przy krawędziach z powodzeniem eksploatowało luki w logice słabszych agentów, jednak w bezpośredniej konfrontacji z człowiekiem ta sztywna i wysoce przewidywalna strategia okazuje się całkowicie nieskuteczna.

5.2.3 Bot stworzony z pomocą uczenia ze wzmocnieniem

Proces uczenia modelu PPO pokazuje, jak agent stopniowo ewoluuje, odrzucając sztywne reguły na rzecz głębokiego zrozumienia topologii planszy i zaawansowanej strategii. Obserwacja preferencji otworzyć (Rysunek 11) i decyzji o zamianie (Rysunek 10) na różnych etapach treningu, poparta bezpośrednimi rozgrywkami botów z autorem, pozwala na wyciągnięcie kilku kluczowych wniosków:

Faza wczesna nauki ($\leq 1.5M$ kroków)

Wczesne modele w dużej mierze opierają się jeszcze na wiedzy zaczerpniętej z fazy Imitation Learning. Widoczne jest to w szerokim, wręcz rozlanym obszarze akceptacji reguły swap (Rysunek 10a i 10b). Choć model ten potrafi już sporadycznie formować mosty, jego zrozumienie tych struktur jest powierzchowne - często nie potrafi ich poprawnie domknąć ani skutecznie obronić się przed podobnymi konstrukcjami ze strony przeciwnika. Z perspektywy zaawansowanego gracza ludzkiego, modele z tej fazy są stosunkowo łatwe do pokonania.

Faza przejściowa i zwrot (2.0M – 2.5M kroków)

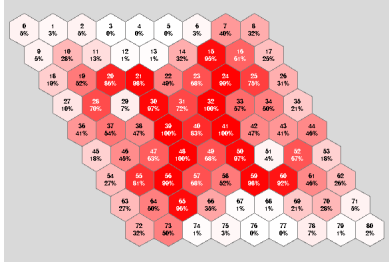
Środkowa faza treningu ukazuje zjawisko tymczasowej destabilizacji polityki agenta. Wersja PPO 2.5M charakteryzuje się najbardziej rozproszonymi decyzjami otwierającymi (Rysunek 11d), co w połączeniu z gorszym wynikiem w turnieju (spadek z 71.99% na 69.72% win-rate) sugeruje okres przejściowy w nauce. Agent na tym etapie odrzucił już proste, bezpieczne heurystyki przyswojone w fazie 0 treningu, jednak nie wykształcił jeszcze w pełni stabilnych mechanizmów defensywnych. W bezpośredniej rozgrywce objawia się to lekkim "pogubieniem" bota 2.5M - choć podejmuje on próby tworzenia zaawansowanych struktur, często nie potrafi się skutecznie bronić na dystans przed atakami stosowanymi przez człowieka.

Faza końcowa (3.0M – 3.5M kroków)

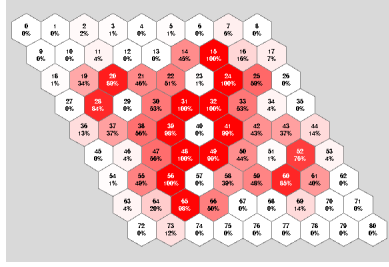
Ostatnie modele prezentują poziom strategiczny, który jest w stanie z powodzeniem rywalizować, a w wielu przypadkach całkowicie zdominować gracza ludzkiego. Jak widać na Rysunkach 10e i 10f, modele 3.0M i 3.5M wypracowały strefę wykonywania zamiany w okolicach przekątnej planszy. W bezpośrednich starciach, PPO 3.0M potrafi już biegle operować blokowaniem na dystans i budowaniem struktur, dzięki czemu mecze z graczem ludzkim (autorem projektu) stają się niezwykle wyrównane. Jednak to wersja 3.5M pokazuje najwyższy poziom. Model ten nie tylko skutecznie blokuje postępy człowieka, ale robi to w taki sposób, że każdy kamień defensywny staje się jednocześnie elementem jego własnej gry ofensywnej. Często prowadzi to do sytuacji, w których gracz ludzki, skupiony na próbach przełamania bloku, nagle orientuje się (często na 3-4 ruchy przed końcem), że sytuacja na planszy jest już matematycznie przegrana, a każda wcześniejsza niedokładność została przez sieć bezwzględnie wykorzystana.

5.3 Porównanie pracochłonności i efektywności podejść

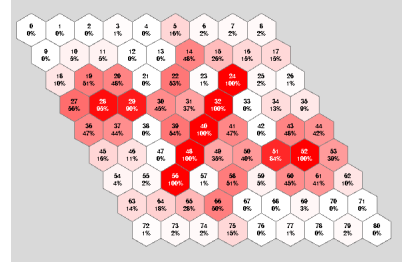
Jedynym kosztem w przypadku agentów heurystycznych był krótki czas na przemyślenie i implementację ich logiki. Osiągają one jednak najslabsze wyniki i zachowują się w sposób wysoce przewidywalny dla oponenta. W podejściu CMA-ES pracochłonność była wysoka ze względu na konieczność ręcznego zaprojektowania funkcji ekstrahujących cechy planszy, ale sam proces optymalizacji wag trwał jednak krótko (ok. 40 minut).



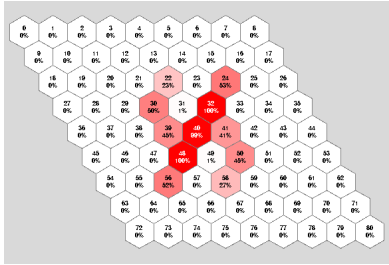
(a) PPO 1.0M



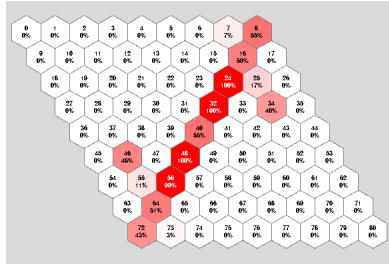
(b) PPO 1.5M



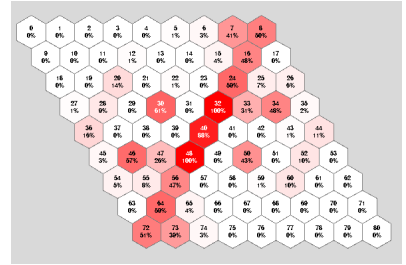
(c) PPO 2.0M



(d) PPO 2.5M

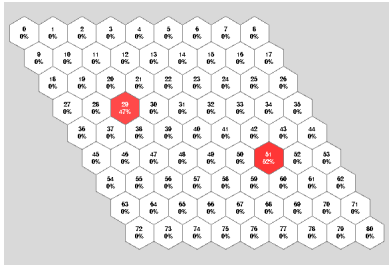


(e) PPO 3.0M

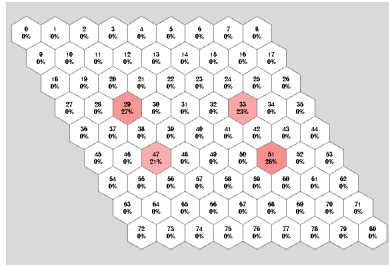


(f) PPO 3.5M

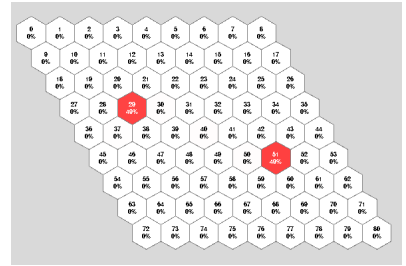
Rysunek 10: Porównanie działania reguły swap w zależności od liczby kroków uczenia modelu PPO.



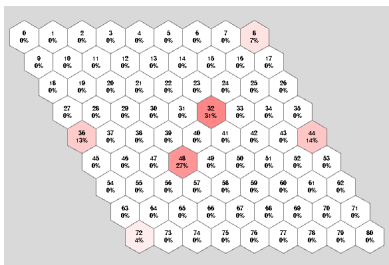
(a) PPO 1.0M



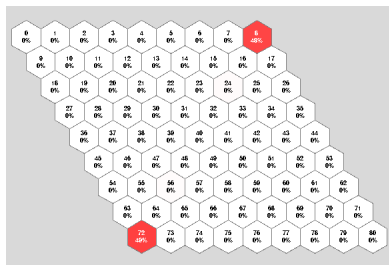
(b) PPO 1.5M



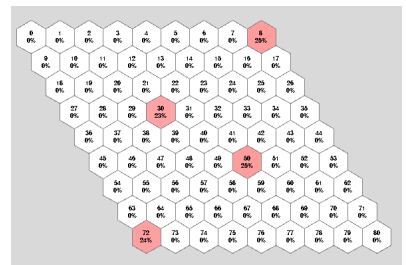
(c) PPO 2.0M



(d) PPO 2.5M



(e) PPO 3.0M



(f) PPO 3.5M

Rysunek 11: Porównanie ruchów otwierających w zależności od liczby kroków uczenia modelu PPO.

Z kolei metoda wykorzystująca uczenie ze wzmocnieniem wymagała największego nakładu pracy (architektura sieci i ligi, faza Imitation Learning) oraz wielogodzinnego procesu trenowania (ok. 5–6 h dla 3.5 mln kroków). Mimo znacznego zużycia zasobów, tylko podejście z użyciem PPO pozwoliło na stworzenie agenta o wysokim poziomie, zdolnego do samodzielnego odkrywania złożonych strategii defensywnych i ofensywnych, które wykraczają poza ramy sztywnej logiki zdefiniowanej w kodzie źródłowym.

Literatura

- [1] HexWiki. Basic Strategy Guide. URL: [https://www.hexwiki.net/index.php/Basic_\(strategy_guide\)](https://www.hexwiki.net/index.php/Basic_(strategy_guide)).
- [2] HexWiki. Draw. URL: <https://www.hexwiki.net/index.php/Draw>.
- [3] HexWiki. Interior template. URL: https://www.hexwiki.net/index.php/Interior_template.
- [4] HexWiki. Swap rule. URL: https://www.hexwiki.net/index.php/Swap_rule.